

Question:

In Azure platform, with a team of 4 people plan a movie recommendation system in a span of 2 sprints which includes complete user stories for all major features, including both functional (e.g., user login, movie browsing, personalised recommendations) and non-functional (e.g., reliability, scalability, performance) requirements.

Portrait the Architecture diagram / Sequence diagram / ER diagram / Class diagram in the Azure platform.

Work on the Test Cases for key functionalities and monitor test results and bug reports in Azure Test Plans. (will be updated after completion of the unit 4 and the unit 5)

MOVIE RECOMMENDATION SYSTEM

EPIC 1: User Registration and Authentication

Feature 1.1: User Registration

User Story 1.1a

As a user, I want to register on the Movie Recommendation System with my credentials using a strong password (minimum 8 characters, 1 numeral and 1 special character), so that I can access my personalised movie recommendations securely.

Task/Bug 1.1a:

- Design registration UI form.

Implement backend validation for strong password.

- Integrate password encryption (hashing).
- Create user table in Azure SQL Database.
- Send email verification link.
- Bug: Password validation not triggering for special characters.
- Bug: Duplicate email registration allowed.

User Story 1.1b

As a user, I want to receive an email verification link after registration so that my account is authenticated and protected from unauthorised access.

Task/Bug 1.1b:

- Configure email service (SMTP/Azure Communication Service).
- Generate unique verification token.
- Set token expiry (24 hours).
- Update account status after verification.
- Bug: Verification link expired immediately.
- Bug: Email sent to spam folder.

Feature 1.2: Login / Logout

User Story 1.2a

As a developer, I want to provide login to the Movie Recommendation System within 3 seconds, so that users can quickly access their personalised content.

Task/Bug 1.2a:

- Optimise database queries.
- Enable caching for user session data.
- Implement JWT-based session management.

- Monitor response time using Azure Monitor.
- Bug: Login takes more than 5 seconds during peak load.
- Bug: Server timeout issue.

User Story 1.2b

As a user, I want to log in to the Movie Recommendation System in a single step, so that I can quickly access my recommendations and watchlist.

Task/Bug 1.2b:

- Design simple login UI.
- Enable remember-me option.
- Implement JWT authentication.
- Provide logout functionality.
- Bug: Session not cleared after logout.
- Bug: Invalid credentials error not displayed properly.

EPIC 2: Movie Browsing and Discovery

Feature 2.1: Browse Movies by Genre / Language

User Story 2.1

As a user, I want to browse movies by genre, language, and release year so that I can discover films that match my taste.

Task/Bug 2.1:

- Build genre and language filter APIs.
- Store movie metadata (genre, language, year) in Azure SQL Database.
- Enable search and filter options on the movie listing page.
- Bug: Genre filter not applying correctly on page reload.

User Story 2.2

As a user, I want to filter movies by rating, language, and release year so that I can find movies that suit my preferences.

Task/Bug 2.2:

- Implement multi-select filter functionality.
- Sort movies by TMDB rating score.
- Add language and genre category tags.
- Bug: Filter state resets after page refresh.

Feature 2.2: Movie Details Page

User Story 2.2.1

As a user, I want to view detailed movie information including poster, cast, synopsis and rating so that I can decide whether to watch it.

Task/Bug 2.2.1:

- Fetch movie poster and metadata from TMDB API.
- Display cast, director, synopsis and TMDB rating.
- Bug: Movie poster not loading for some titles.

User Story 2.2.2

As a user, I want to see if a movie is available on popular streaming platforms so that I know where to watch it.

Task/Bug 2.2.2:

- Add streaming availability field in database.

- Display streaming platform badges (e.g. Netflix, Prime).
- Bug: Incorrect streaming platform shown for some movies.

EPIC 3: Personalised Movie Recommendations

Feature 3.1: Recommendation Engine

User Story 3.1

As a user, I want to receive movie recommendations based on my watch history and preferences so that I discover films I will enjoy.

Task/Bug 3.1:

- Build collaborative filtering recommendation model.
- Store user ratings and watch history in database.
- Generate top-10 personalised recommendations per user.
- Bug: Recommendations not updating after new ratings are added.

User Story 3.2

As a user, I want to receive mood-based movie recommendations so that I can find films suited to how I feel right now.

Task/Bug 3.2:

- Build mood selection UI (e.g. Happy, Thriller, Romantic).
- Map mood tags to movie genres in the backend.
- Return filtered recommendations based on selected mood.
- Bug: Mood filter returning irrelevant genres.

Feature 3.2: Similar Movie Suggestions

User Story 3.2.1

As a user, I want to see a list of similar movies on any movie's detail page so that I can find related content easily.

Task/Bug 3.2.1:

- Implement content-based filtering using genre and cast similarity.
- Fetch similar movie data from TMDB API.
- Display similar movies as a horizontal carousel on the detail page.
- Bug: Similar movies list showing unrelated titles.

User Story 3.2.2

As a user, I want to receive weekly email recommendations based on my recent activity so that I stay updated on new films.

Task/Bug 3.2.2:

- Build a scheduled Azure Function to send weekly recommendation emails.
- Update order status to "Paid".
- Bug: Email not triggered for users with no recent activity.

EPIC 4: Watchlist and Viewing History

Feature 4.1: Watchlist Management

User Story 4.1

As a user, I want to add and remove movies from my watchlist so that I can keep track of films I plan to watch.

Task/Bug 4.1:

- Create watchlist table in Azure SQL Database.

- Build add/remove watchlist API endpoints.
- Display watchlist on user profile page.
- Bug: Watchlist not persisting after session logout.

User Story 4.2

As a user, I want to rate and review movies I have watched so that I can share my opinion and improve my recommendations.

Task/Bug 4.2:

- Build star-rating and review submission UI.
- Store ratings and reviews in the database.
- Bug: Rating submission failing silently for some users.

Feature 4.2: Viewing History

User Story 4.2.1

As a user, I want to view my complete movie-watching history so that I can revisit films I have seen before.

Task/Bug 4.2.1:

- Log each movie viewed to the history table in the database.
- Display watch history with dates on user profile.
- Bug: Duplicate entries appearing in watch history.

User Story 4.2.2

As a user, I want to mark movies as “Watched” from the recommendation page so that the system can refine future suggestions.

Task/Bug 4.2.2:

- Build “Mark as Watched” button on movie cards.
- Update user history and re-trigger recommendation recalculation.
- Bug: Watched status not reflected immediately on UI.

EPIC 5: Admin Dashboard and System Management

Feature 5.1: User Feedback and Ratings Management

User Story 5.1

As a user, I want to provide ratings and written feedback on movies so that I can contribute to the community and help others decide.

Task/Bug 5.1:

- Build feedback form with star rating and text comment field.
- Store ratings and reviews in database linked to user and movie.
- Calculate and display average rating per movie.
- Bug: Average rating not updating after new submissions.

User Story 5.2

As an admin, I want to review and moderate user reviews so that inappropriate content is removed and service quality is maintained.

Task/Bug 5.2:

- Build admin review moderation panel in Azure DevOps.
- Flag and filter reviews below a quality threshold.
- Bug: Reviews not displaying in chronological order.

Feature 5.2: Movie Data and Report Management

User Story 5.2.1

As an admin, I want to sync the latest movie data from TMDB API on a scheduled basis so that the platform always shows up-to-date content.

Task/Bug 5.2.1:

- CRUD operations for menu.
- Update price & availability.
- Bug: Changes not reflected immediately.

User Story 5.2.2

As an admin, I want to generate sales reports, so that I can analyze business performance.

Task/Bug 5.2.2:

- Build Azure Function to fetch new movie data daily from TMDB API.
- Export platform usage reports as PDF or Excel.
- Bug: Scheduled sync job failing silently on API timeout.

MOVIE RECOMMENDATION SYSTEM USING MICROSOFT AZURE

(Agile – 2 Sprints | Team of 4 | Architecture + Test Plans)

Agile Planning (2 Sprints)

Team Structure (4 Members)

Role	Responsibility
Scrum Master	Sprint planning, daily standups, removing blockers
Backend Developer	API, database, recommendation engine logic
Frontend Developer	UI (Web/App), user experience, movie browsing pages
QA/Test Engineer	Test cases, Azure Test Plans, bug tracking

Sprint Planning Overview

Sprint 1 (Week 1–2)

Goal: Core user and movie browsing functionalities

Features:

- User Registration & Login
- Movie listing with genre and language filters
- Movie detail page with TMDB metadata
- Watchlist management (add/remove)

- User ratings and review submission
- Basic recommendation engine (genre-based)
- Azure SQL Database setup (Users, Movies, Ratings, Watchlist)
- Deployment on Azure App Service

Sprint 2 (Week 3–4)

Goal: Advanced recommendations & Quality

Features:

- Personalised recommendation engine (collaborative filtering)
- Admin dashboard for data and review management
- Viewing history and watched status tracking
- Weekly email recommendations via Azure Functions
- Performance optimisation and caching
- Security enhancements (rate limiting, input validation)
- Load testing with Azure Load Testing
- Bug fixing
- CI/CD pipeline setup

Architecture Diagram:

flowchart TD

User[User - Web / Mobile App] -->|HTTPS Request| AppService[Azure App Service - Flask API]

AppService --> Auth[Azure Active Directory - JWT Auth]

AppService --> DB[(Azure SQL Database)]

AppService --> TMDB[TMDB API - Movie Metadata]

AppService --> RecEngine[Recommendation Engine - Python]

AppService --> AzFunc[Azure Functions - Weekly Email]

AppService --> Monitor[Azure Monitor]

AppService --> DevOps[Azure DevOps CI/CD]

DevOps --> AppService

subgraph Azure Cloud

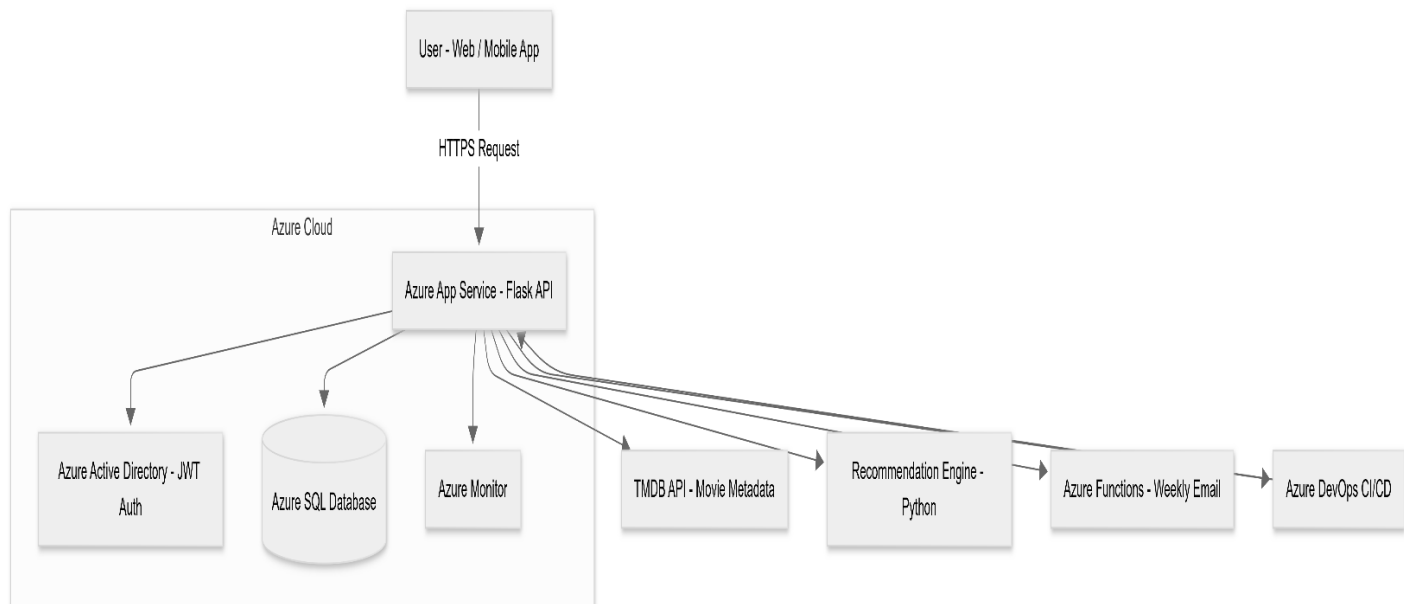
AppService

DB

Auth

Monitor

end



Sequence Diagram

sequenceDiagram

participant User

participant UI as Web App

participant API as Flask API

participant DB as Azure SQL

participant TMDB as TMDB API

participant Rec as Recommendation Engine

User->>UI: Login

UI->>API: Authenticate Request

API->>DB: Validate Credentials

DB-->>API: User Record Found

API-->>UI: JWT Token Returned

User->>UI: Browse Movies / Apply Filters

UI->>API: GET /movies?genre=Action

API->>TMDB: Fetch Movie Metadata

TMDB-->>API: Movie List Returned

API-->>UI: Display Movie Cards

User->>UI: Request Recommendations

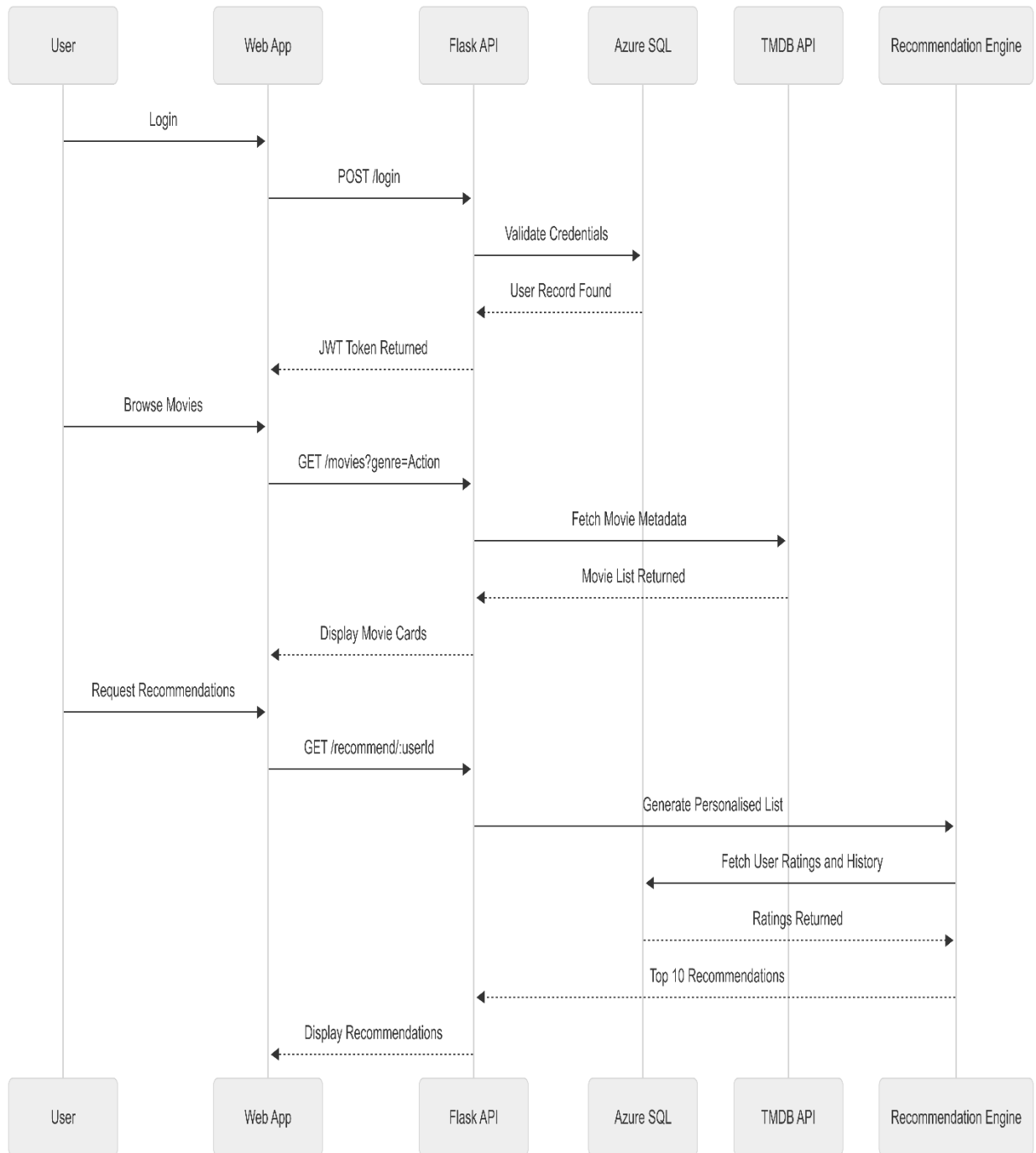
UI->>API: GET /recommend/:userId

API->>Rec: Generate Personalised List

Rec->>DB: Fetch User Ratings and History

DB-->>Rec: Ratings Returned

Rec-->>API: Top 10 Movie Recommendations



Class Diagram

API
classDiagram

```
class User {  
    +userId: String  
    +name: String  
    +email: String  
    +password: String  
    +watchlist: List  
    +login()  
    +logout()  
}
```

```
class Movie {  
    +movieId: String  
    +title: String  
    +genre: String  
    +rating: Float  
    +cast: List  
    +getDetails()  
    +getSimilar()  
}
```

```
class Rating {  
    +ratingId: String  
    +score: Int  
    +review: String  
    +createdAt: Date  
    +submit()  
}
```

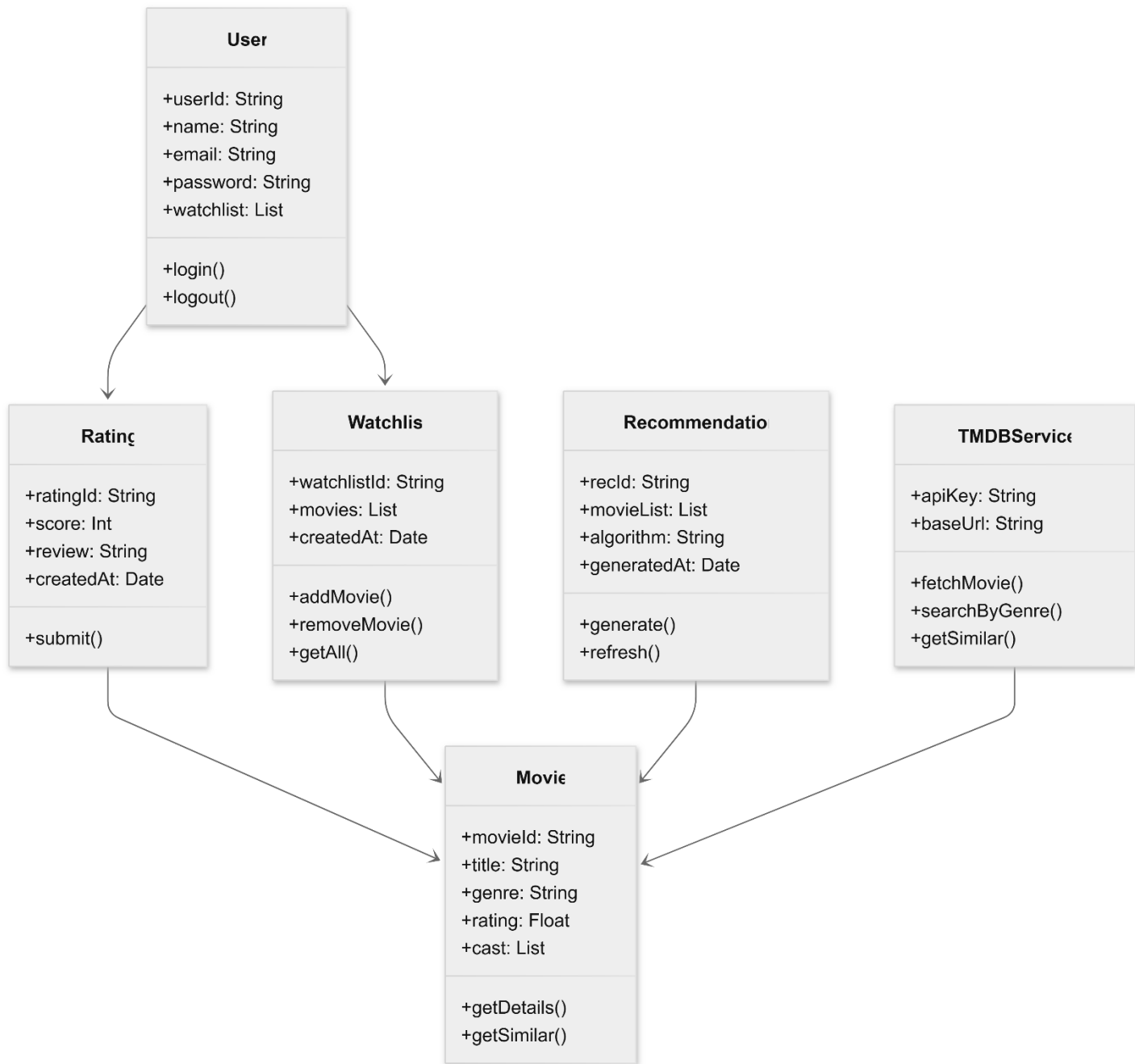
```
class Watchlist {
```

```
    +watchlistId: String  
    +movies: List  
    +createdAt: Date  
    +addMovie()  
    +removeMovie()  
    +getAll()  
}
```

```
class Recommendation {  
    +recId: String  
    +movieList: List  
    +algorithm: String  
    +generatedAt: Date  
    +generate()  
    +refresh()  
}
```

```
class TMDBService {  
    +apiKey: String  
    +baseUrl: String  
    +fetchMovie()  
    +searchByGenre()  
    +getSimilar()  
}
```

User --> Rating
User --> Watchlist
Rating --> Movie
Watchlist --> Movie
Recommendation --> Movie
TMDBService --> Movie



ER Diagram

A. Crow's Foot style:

- Entities are shown as boxes
- Attributes are listed inside
- Relationships use symbols like ||--o{

erDiagram

```
USER {
    int user_id PK
    string name
    string email
    string password_hash
}
```

```
MOVIE {
    int movie_id PK
    string title
    string genre
    float tmdb_rating
}
```

```
RATING {
    int rating_id PK
    int user_id FK
    int movie_id FK
    int score
    string review
}
```

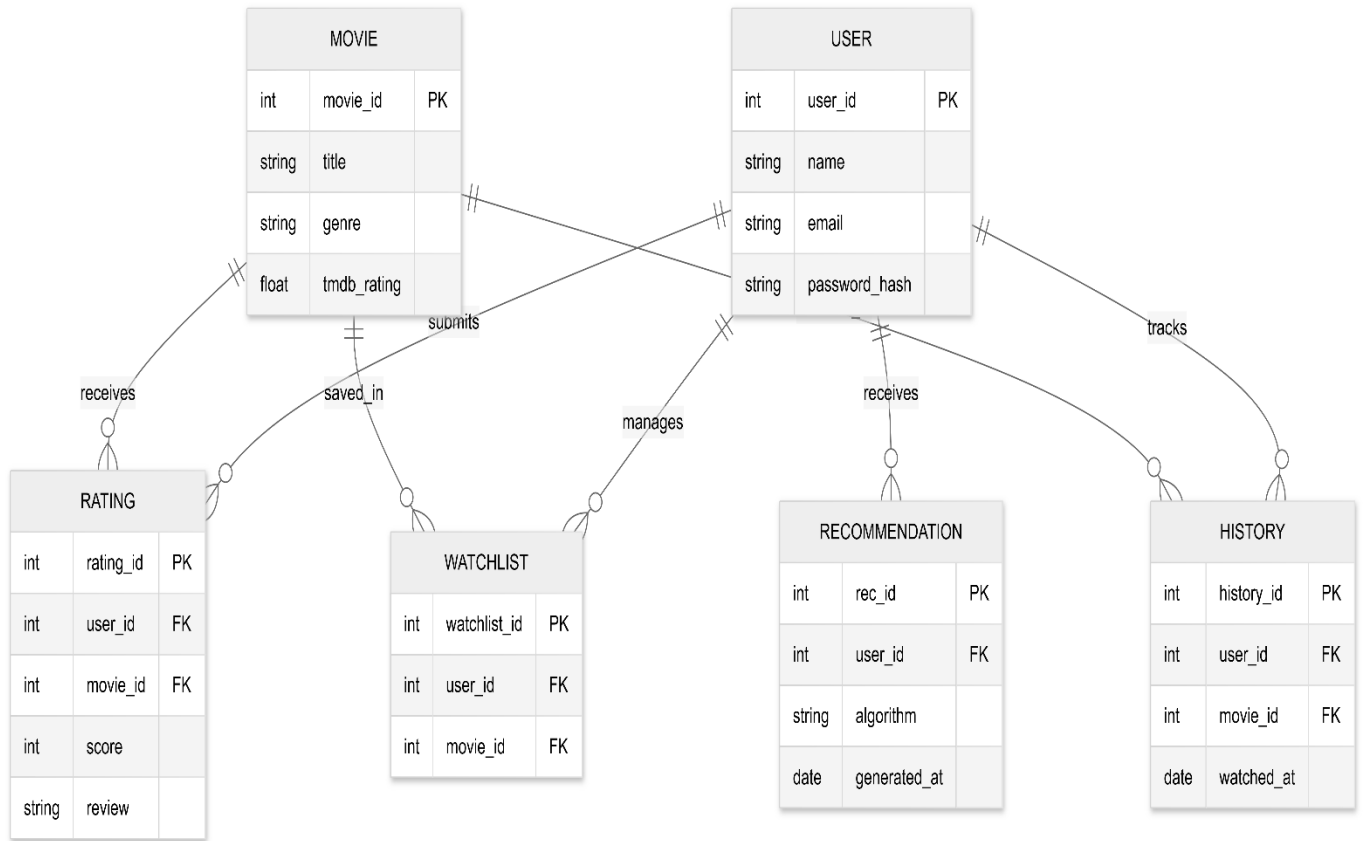
```
WATCHLIST {
    int watchlist_id PK
```

```
    int user_id FK
    int movie_id FK
}
```

```
HISTORY {
    int history_id PK
    int user_id FK
    int movie_id FK
    date watched_at
}
```

```
RECOMMENDATION {
    int rec_id PK
    int user_id FK
    string algorithm
    date generated_at
}
```

```
USER ||--o{ RATING : submits
USER ||--o{ WATCHLIST : manages
USER ||--o{ HISTORY : tracks
MOVIE ||--o{ RATING : receives
MOVIE ||--o{ WATCHLIST : saved_in
MOVIE ||--o{ HISTORY : watched_in
USER ||--o{ RECOMMENDATION : receives
```



B. DBMS format -**Chen notation**,

-  Entity → Rectangle
-  Relationship → Diamond
-  Attributes → Ovals (circles)
-  Primary Key → Underlined
- Lines connect entity → relationship → entity
-

flowchart LR

%% Entities

User[USER]

Movie[MOVIE]

Rating[RATING]

Watchlist[WATCHLIST]

History[HISTORY]

Recommendation[RECOMMENDATION]

%% Relationships (Diamonds)

Submits{SUBMITS}

Receives{RECEIVES}

Manages{MANAGES}

Tracks{TRACKS}

Gets{GETS}

%% Connections - mirroring the chained structure

User --> Submits

Submits --> Rating

Rating --> Receives

Receives --> Movie

User --> Manages

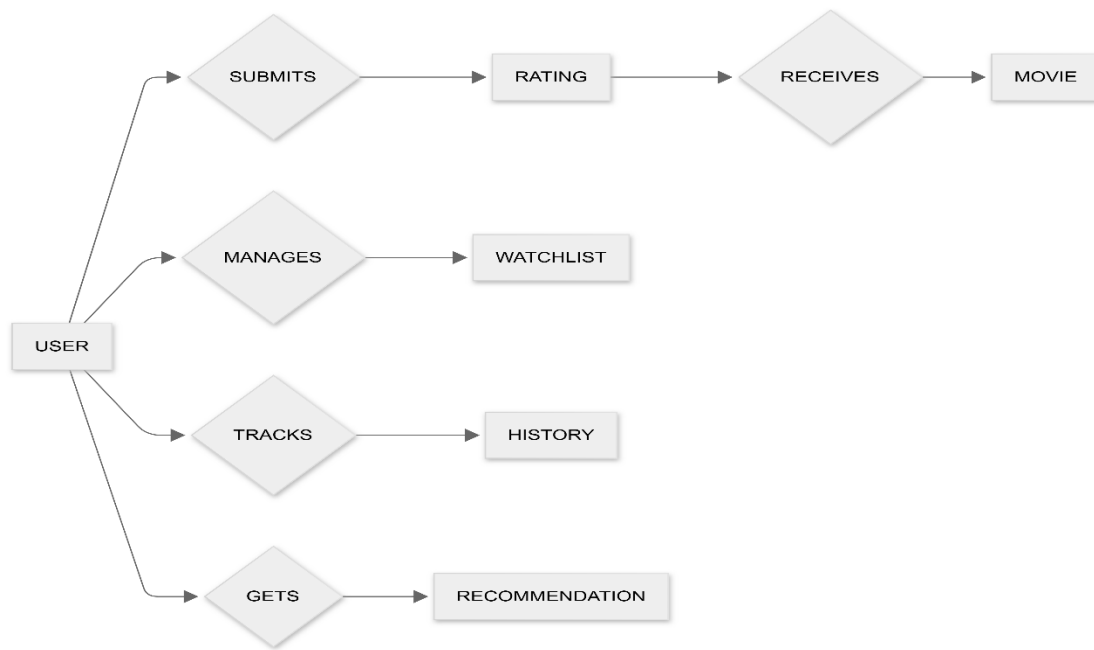
Manages --> Watchlist

User --> Tracks

Tracks --> History

User --> Gets

Gets --> Recommendation



```

SyncTMDB[Sync TMDB Movie Data]
end

```

Use case diagram:

flowchart LR

%% Actors

User((User))

Admin((Admin))

RecEngine((Recommendation Engine))

TMDBAPI((TMDB API))

AzureFunc((Azure Functions))

%% System Boundary

subgraph Movie Recommendation System

Register[Register]

Login[Login / Logout]

Browse[Browse Movies]

ViewMovie[View Movie Details]

AddWatchlist[Add to Watchlist]

RateMovie[Rate and Review Movie]

GetRecs[Get Personalised

Recommendations]

ViewHistory[View Watch History]

MoodRec[Mood-based Recommendations]

ManageMovies[Manage Movie Data]

ModerateReviews[Moderate User Reviews]

ViewReports[Generate Usage Reports]

%% User actions

User --> Register

User --> Login

User --> Browse

User --> ViewMovie

User --> AddWatchlist

User --> RateMovie

User --> GetRecs

User --> ViewHistory

User --> MoodRec

%% Admin actions

Admin --> ManageMovies

Admin --> ModerateReviews

Admin --> ViewReports

%% System actions

RecEngine --> GetRecs

AzureFunc --> SyncTMDB

SyncTMDB --> TMDBAPI

